Fine-Tuning Reshuffles Neighbourhoods Fast, but Not on Schedule: Step-Resolved Embedding Drift Across Scale, Domain, and Recipe

NeuriCo

Abstract

Domain fine-tuning erodes the general abilities of pretrained language models, and benchmarking every checkpoint to catch this is expensive. Measuring how the geometry of representations drifts on a fixed probe set is cheap and needs no labels, but it is unclear how fast that drift happens, what controls its speed, and whether it signals forgetting. We test four pre-registered claims: sub-1B models lose half their nearest-neighbour structure within 500 steps, isotropy declines monotonically, benchmark accuracy starts to fall once preservation drops below 60%, and medical fine-tuning preserves more structure than code. We track 22 LoRA and full fine-tuning runs of five base models (160M–3B parameters, Qwen2.5 and Pythia) on medical, code, and general text at 17 checkpoints up to 3,000 steps. None of the four claims holds as stated. Larger models drift less at matched steps (medical, step 500: 0.42 for 160M vs. 0.85 for 3B), but the learning rate matters more than size: the same 0.5B model ends at 0.73, 0.57, or 0.22 preservation as the learning rate rises from 2e-4 to 1e-3. Every run that fell below 0.5 also had rising in-domain loss. No run becomes monotonically less isotropic. In Qwen models, benchmark forgetting begins while preservation is still 0.77–0.90. Code fine-tuning preserves more general-text geometry than medical fine-tuning (by 0.043, $p = 3 \times 10^{-12}$), even though code shares less vocabulary with general text. Preservation tracks damage to general-text language modelling closely (Spearman $\rho = -0.90$), so it is a useful sign of unstable training. It does not provide a universal forgetting threshold.

1 Introduction

Fine-tuning a pretrained language model on a narrow domain is the standard way to specialise it. It also erodes the model's general abilities, a problem known as catastrophic forgetting [McCloskey and Cohen, 1989, Luo et al., 2023]. Practitioners who want to stop training before too much is lost must evaluate general benchmarks at every checkpoint, which is slow and needs labelled data. A cheaper option is to watch the model's *representation geometry* on a fixed set of probe texts. If a pretrained model's neighbourhood structure collapses before or while it forgets, a label-free geometric monitor could replace part of the benchmark suite.

How fast does fine-tuning destroy pretraining geometry? Prior work shows that fine-tuning changes representations [Merchant et al., 2020], that contextual embeddings are anisotropic [Ethayarajh, 2019, Mu and Viswanath, 2018], and that forgetting grows with the amount of fine-tuning and shrinks with model size [Kalajdzievski, 2024, Biderman et al., 2024]. These studies mostly report end states or global similarity scores. They leave open three practical questions. First, how many steps does it take for local neighbourhood structure to fall apart, and does this depend on

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

scale? Second, does a geometric quantity cross a fixed threshold when forgetting begins? Third, does the fine-tuning domain matter, and is lexical closeness to pretraining data a good predictor?

We turn these questions into four specific hypotheses. **H1**: kNN@10 preservation of general-text probes falls below 50% within 500 steps for sub-1B models but only after 2,000 steps for 7B models. **H2**: isotropy decreases monotonically. **H3**: general-knowledge accuracy starts to decline once preservation drops below 60%. **H4**: medical fine-tuning preserves more structure than code fine-tuning because medical text shares more vocabulary with pretraining data.

We test them with a step-resolved study (figure 1). We fine-tune five base models (Pythia-160M, Pythia-410M, Qwen2.5-0.5B, Qwen2.5-1.5B, Qwen2.5-3B) on synthetic clinical notes, Python code, and a WIKITEXT-103 control, and evaluate 17 checkpoints between step 0 and step 3,000. At each checkpoint we measure kNN neighbourhood preservation, CKA, and three isotropy measures on 1,000 held-out probes. We also measure general knowledge on 1,070 MMLU and ARC-EASY items with item-level paired tests, and perplexity on general and in-domain text. Learning-rate and full fine-tuning ablations separate the effect of model size from the effect of update size.

None of the four hypotheses holds as stated. Within each model family, larger models do keep more of their geometry at matched steps: after 500 medical steps, preservation is 0.42, 0.74, 0.79, 0.82, and 0.85 for the 160M, 410M, 0.5B, 1.5B, and 3B models. But Qwen2.5-0.5B never falls below 0.71 under the standard recipe, while the same model reaches 0.22 when the learning rate is 1e-3. All seven runs that ended below 0.5 had rising held-out in-domain loss. No run became monotonically less isotropic on any of three measures. In Qwen models, significant benchmark forgetting begins at preservation 0.77–0.90, far above 0.6. Medical fine-tuning preserves more general-text geometry than code fine-tuning at only 17% of matched checkpoints. This happens even though clinical notes are lexically much closer to general text (unigram JSD 0.45 vs. 0.63 bits). Preservation is still useful: it tracks damage to general-text language modelling with Spearman $\rho = -0.90$, and a sharp early drop reliably flags unstable training.

Our contributions are:

- We conduct a step-resolved study of embedding drift across 22 fine-tuning runs, five model sizes, two families, three domains, and two training methods, with item-level forgetting tests at every checkpoint.
- We show that the speed of neighbourhood collapse depends more on the learning rate and on training stability than on model size, and that collapse below 50% happens only in runs whose in-domain loss rises.
- We find that global isotropy is a weak, non-monotone drift signal, and that no fixed preservation threshold marks the start of benchmark forgetting.
- We find that vocabulary overlap does not predict geometric preservation: code fine-tuning preserves more general-text geometry than medical fine-tuning, and we identify the confound (how much there is to learn) that a follow-up design must remove.

Section 2 reviews related work. Section 3 describes the models, data, and metrics. Section 4 tests each hypothesis. Section 5 discusses implications and limitations, and section 6 concludes.

2 Related Work

Representation change under fine-tuning. Merchant et al. [2020] showed that fine-tuning BERT mostly changes the upper layers and leaves lower layers close to the pretrained model. Representation similarity is usually measured globally, with SVCCA [Raghu et al., 2017] or centred kernel alignment [Kornblith et al., 2019]. Global measures can hide large changes in local structure, because a near-rotation of the space keeps CKA high while neighbours change. We therefore focus on kNN neighbourhood preservation, a local measure common in dimensionality-reduction evaluation [Venna and Kaski, 2006], and report CKA alongside it. Unlike earlier end-of-training comparisons, we follow the full trajectory over 17 checkpoints.

Anisotropy of contextual embeddings. Contextual representations occupy a narrow cone [Ethayarajh, 2019], and removing dominant directions improves static embeddings [Mu and Viswanath, 2018]. Rudman et al. [2022] proposed IsoScore to measure how uniformly a space uses its dimensions. These works describe the isotropy of a trained model. We ask whether fine-tuning moves isotropy in a consistent direction, and find that it does not.

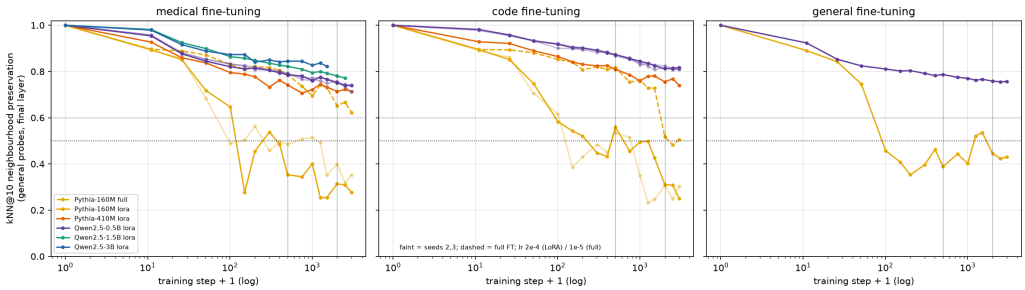


Figure 1: kNN@10 preservation of 1,000 general-text probes (final layer) against training step for medical, code, and general fine-tuning. Faint lines are additional seeds; dashed lines are full fine-tuning of Pythia-160M. Most of the loss happens in the first $\sim$100 steps. Larger Qwen models drift less, but only Pythia-160M under LoRA falls below 0.5 (dotted line) with the standard recipe, and all those runs diverged in in-domain loss.

Forgetting in language-model fine-tuning. Catastrophic forgetting has been studied since early connectionist work [McCloskey and Cohen, 1989, Kirkpatrick et al., 2017]. For LLMs, Luo et al. [2023] measured forgetting during continual instruction tuning, Kalajdzievski [2024] fitted scaling laws relating forgetting to fine-tuning loss and parameter count, and Biderman et al. [2024] showed that LoRA [Hu et al., 2022] learns less and forgets less than full fine-tuning. Our learning-rate and full fine-tuning ablations agree that update size, not only scale, sets how much is lost. We add a link between local geometry and item-level forgetting, and show that the geometry-to-forgetting mapping differs across model families.

Domain distance and transfer. Domain-adaptive pretraining work [Gururangan et al., 2020] uses vocabulary overlap to describe how far a domain is from pretraining data. We test whether this distance predicts how much general-text geometry fine-tuning preserves, and find the opposite of the intuitive prediction.

3 Method

3.1 Measuring geometric drift

Let f_t be the model after t fine-tuning steps and $\{x_i\}_{i=1}^{N}$ a fixed probe set. We represent each probe by $\boldsymbol{h}_i^t$, the mean of the hidden states at positions ≥ 1 of a given layer. We skip position 0 because it acts as an attention sink. We use the final (post-norm) layer as the primary view and the middle layer as a robustness check. All metrics compare step t with step 0.

kNN preservation. Let $N_k^t(i)$ be the k nearest neighbours of probe i under cosine similarity at step t. Preservation is

$$P_k(t) = \frac{1}{N} \sum_{i=1}^{N} \frac{|N_k^0(i) \cap N_k^t(i)|}{k}. \tag{1}$$

$P_k(t) = 1$ means every probe keeps all its neighbours. Our primary metric is P_{10}; we also report $k \in \{5, 20\}$ and a mean-centred variant that removes the effect of a shared offset.

Global similarity and isotropy. We compute linear CKA [Kornblith et al., 2019] between $\boldsymbol{H}^0$ and $\boldsymbol{H}^t$. We measure isotropy in three ways: mean pairwise cosine (higher means more anisotropic), IsoScore [Rudman et al., 2022], and effective rank, the exponential of the entropy of the normalised covariance eigenvalues.

Forgetting. General knowledge (GK) is the pooled accuracy on 1,070 items: 570 MMLU questions [Hendrycks et al., 2021] (10 per subject, 0-shot, argmax over the log-probabilities of " A"–" D") and 500 ARC-EASY test questions [Clark et al., 2018] (length-normalised accuracy). We also measure per-token NLL on 64×512 held-out tokens of WIKITEXT-103 [Merity et al., 2016] and of the in-domain corpus.

3.2 Models and training

We use base checkpoints from two families: Qwen2.5-0.5B, Qwen2.5-1.5B, Qwen2.5-3B [Qwen Team, 2024], which share a tokenizer and pretraining corpus, and Pythia-160M and Pythia-410M [Biderman et al., 2023], trained on the Pile. The standard recipe is identical for every size, so that steps are comparable. It applies LoRA [Hu et al., 2022] (rank 64, $\alpha = 128$, no dropout) to every linear layer and trains with AdamW ($\beta = (0.9, 0.95)$, lr 2e-4, 50 warm-up steps then constant, gradient clipping at 1.0, no weight decay). Each step uses a batch of 4×512 tokens, so 3,000 steps cover 6.1M tokens. Weights are in bf16 with gradient checkpointing.

Ablations. To separate model size from update size, we train Qwen2.5-0.5B with LoRA at lr 5e-4 and 1e-3, and Pythia-160M with full fine-tuning (fp32 master weights, bf16 autocast, lr 1e-5). We run three seeds for Qwen2.5-0.5B medical and code, two for Pythia-160M medical and code, and one elsewhere; the seed sets document order and LoRA initialisation. In total we analyse 22 runs (table 1).

3.3 Data

Fine-tuning corpora. Medical: 40k synthetic clinical notes from ASCLEPIUS [Kweon et al., 2024]. Code: 30k Python files from CODEPARROT-clean-valid (800–20k characters, not auto-generated). General (control): WIKITEXT-103 training paragraphs. The control separates the effect of being fine-tuned at all from the effect of domain shift. Texts are joined with EOS and cut into 512-token chunks.

Probes. The primary probe set has 1,000 WIKITEXT-103 test and validation paragraphs truncated to 128 tokens. Secondary sets have 300 held-out medical and 300 held-out code documents. All probe and evaluation data are disjoint from training data.

Vocabulary overlap. To test the premise of H4, we compare the Qwen-token unigram distribution of $\sim$2M tokens per corpus with WIKITEXT-103, using Jensen–Shannon divergence (JSD), the token mass inside the top 10k WIKITEXT-103 tokens, and word-type coverage.

3.4 Evaluation protocol and statistics

We evaluate 17 checkpoints, at steps 0, 10, 25, 50, 100, 150, 200, 300, 400, 500, 750, 1,000, 1,250, 1,500, 2,000, 2,500, and 3,000. We estimate threshold-crossing steps by linear interpolation in $\log(1 + t)$. For forgetting, we run an exact McNemar test on the 1,070 items, comparing each checkpoint with step 0. For H2 we compute Spearman correlations with step and count monotonicity violations. For H3 we compare checkpoints above and below preservation 0.6 with a Mann–Whitney U test, and report Spearman correlations. For H4 we use a paired Wilcoxon test over checkpoints matched on model, method, learning rate, seed, and step.

Training health. We label a run as *learned* if its final in-domain NLL is more than 0.05 nats below the step-0 value, *diverged* if it is more than 0.25 nats above, and *no gain* otherwise. This label turns out to be central to interpreting H1.

Compute. All runs used 4 shared RTX A6000 GPUs, usually with 2–6 GB free per GPU. A Pythia-160M run took 7–10 minutes and a Qwen2.5-0.5B run 20–26 minutes. The Qwen2.5-1.5B and Qwen2.5-3B medical runs were stopped at steps 2,500 and 1,500 because of GPU contention. A 7B run started but ran out of memory when another job on the same GPU grew (section 5.1).

4 Results

4.1 H1: Bigger models drift less, but the recipe sets the speed

Table 1 and figure 2 show kNN@10 preservation of the general-text probes over training.

Scale helps at matched steps. Within each family, preservation rises with size at every matched step (figure 2, left). After 1,000 medical steps, Qwen preservation is 0.76, 0.80, and 0.83 for 0.5B, 1.5B, and 3B. Pythia-410M is far more stable than Pythia-160M (0.74 vs. 0.42 at step 500). The direction of H1 therefore holds.

Table 1: kNN@10 preservation of general-text probes (final layer) by step. Values are means over seeds; the standard deviation across the three Qwen2.5-0.5B seeds is ≤ 0.016. "Cross" is the interpolated step at which preservation first falls below 0.5. "Health" compares final with step-0 held-out in-domain NLL. Every run that ends below 0.5 is labelled *diverged*. Highest preservation per column among standard-recipe medical runs in **bold**.

Model	Recipe	Domain	10	100	500	1000	2000	3000	Cross	Health
Pythia-160M (2 seeds)	LoRA 2e-4	medical	0.89	0.57	0.42	0.46	0.36	0.31	∼96–117	diverged
Pythia-160M (2 seeds)	LoRA 2e-4	code	0.89	0.60	0.55	0.42	0.31	0.28	∼123–224	diverged
Pythia-160M	LoRA 2e-4	general	0.89	0.46	0.39	0.40	0.45	0.43	∼90	diverged
Pythia-160M	full 1e-5	medical	0.90	0.83	0.79	0.70	0.65	0.62	never	learned
Pythia-160M	full 1e-5	code	0.90	0.85	0.82	0.76	0.52	0.51	(∼2,240)	no gain
Pythia-410M	LoRA 2e-4	medical	0.93	0.80	0.74	0.72	0.71	0.71	never	learned
Pythia-410M	LoRA 2e-4	code	0.93	0.87	0.81	0.76	0.76	0.74	never	diverged
Qwen2.5-0.5B (3 seeds)	LoRA 2e-4	medical	0.96	0.83	0.79	0.76	0.75	0.73	never	learned
Qwen2.5-0.5B (3 seeds)	LoRA 2e-4	code	0.98	0.91	0.87	0.84	0.82	0.81	never	no gain
Qwen2.5-0.5B	LoRA 2e-4	general	0.92	0.81	0.79	0.77	0.76	0.76	never	learned
Qwen2.5-0.5B	LoRA 5e-4	medical	0.91	0.78	0.70	0.65	0.58	0.57	never	learned
Qwen2.5-0.5B	LoRA 5e-4	code	0.96	0.86	0.77	0.73	0.66	0.63	never	diverged
Qwen2.5-0.5B	LoRA 1e-3	medical	0.88	0.71	0.43	0.35	0.25	0.22	∼420	diverged
Qwen2.5-0.5B	LoRA 1e-3	code	0.95	0.78	0.50	0.37	0.28	0.27	∼500	diverged
Qwen2.5-1.5B	LoRA 2e-4	medical	**0.98**	0.86	0.82	0.80	**0.78**	(0.77 @2,500)	never	learned
Qwen2.5-3B	LoRA 2e-4	medical	**0.98**	**0.87**	**0.85**	**0.83**	–	(0.82 @1,500)	never	learned

Table 2: Isotropy change over 22 runs (final layer, general probes). No run moves monotonically toward anisotropy on any measure, and the median net change is a few percent.

Measure	Monotone	Net more anisotropic	Median change
Mean pairwise cosine (↑ anisotropic)	0 / 22	14 / 22	$+2.3\%$
IsoScore (↓ anisotropic)	0 / 22	18 / 22	-8.3%
Effective rank (↓ anisotropic)	0 / 22	17 / 22	-5.7%

But sub-1B is not enough for collapse. Qwen2.5-0.5B (494M parameters) never falls below 0.71 in 3,000 steps under the standard recipe. Only Pythia-160M crosses 0.5 within 500 steps, at roughly steps 90–220.

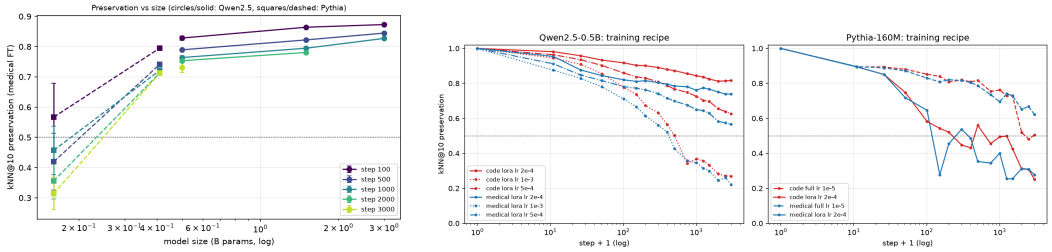
The learning rate matters more than size. Raising the LoRA learning rate on Qwen2.5-0.5B from 2e-4 to 5e-4 to 1e-3 lowers final medical preservation from 0.73 to 0.57 to 0.22 (figure 2, right). At lr 1e-3, Qwen2.5-0.5B crosses 0.5 at steps ∼420–500, which matches the H1 prediction. For Pythia-160M, full fine-tuning at lr 1e-5 keeps 0.62 after 3,000 medical steps, while LoRA at lr 2e-4 falls to 0.31. A fivefold change in learning rate on one model moves final preservation by 0.51 (0.73 $\rightarrow$ 0.22), while a sixfold change in Qwen size moves step-500 preservation by only 0.06 (0.79 $\rightarrow$ 0.85).

Collapse goes together with failed optimisation. All seven runs that end below 0.5 have *rising* held-out in-domain NLL. Pythia-160M LoRA runs lose 0.5–2.1 nats in every domain, including on WikiText-103 itself, and Qwen2.5-0.5B at lr 1e-3 loses 0.4 (medical) and 1.5 (code) nats. Runs that actually learn their domain end between 0.57 and 0.82. Fast loss of pretraining geometry in these runs is therefore a symptom of destructive optimisation, not of successful specialisation.

Most drift happens early. In all runs, most of the change happens in the first ∼100 steps. After that, curves decline slowly on a log-step axis (figure 1).

Fine-tuning itself, not the domain, causes most drift. The WikiText-103 control drifts about as much as domain fine-tuning: Qwen2.5-0.5B ends at 0.76 after general fine-tuning vs. 0.73 after medical fine-tuning.

Robustness. $k = 5$ and $k = 20$ give the same ordering of runs; $k = 20$ values are 0.01–0.05 higher. The mean-centred variant is on average 0.024 higher (at most 0.12) with the same ordering. The middle layer is usually somewhat more stable (median $+0.04$) and shows the same pattern.



(a) Preservation vs. model size

(b) Recipe ablations

Figure 2: **(a)** Medical fine-tuning, standard LoRA recipe: preservation grows with size at each step within each family (squares: Pythia; circles: Qwen2.5; error bars over seeds). **(b)** Changing the recipe on a single model moves preservation much more than size does. For Qwen2.5-0.5B, lr 1e-3 drives preservation to ∼0.25, and for Pythia-160M, full fine-tuning at lr 1e-5 (dashed) drifts far less than LoRA at 2e-4.

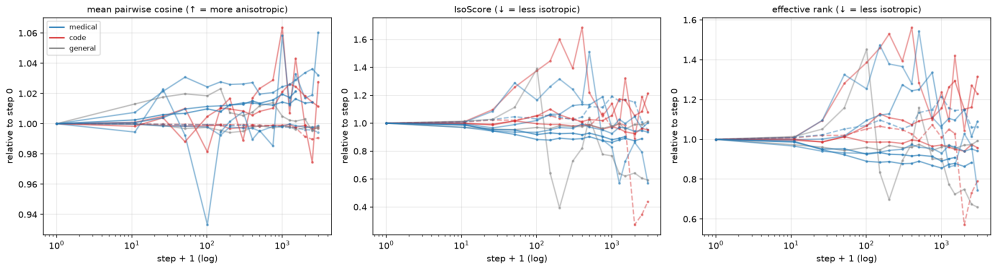


Figure 3: Three isotropy measures relative to step 0 for all runs (blue: medical; red: code; grey: general; dashed: full fine-tuning). Trajectories are noisy and non-monotone, and net changes are small compared with the loss of neighbourhood structure in figure 1.

4.2 H2: Isotropy does not fall monotonically

Table 2 and figure 3 summarise isotropy. Most runs end slightly more anisotropic than they started, but none gets there monotonically. Pythia-160M LoRA runs first become much *more* isotropic (IsoScore up to 1.6× its step-0 value at steps 100–500) and then fall back. Meanwhile, local neighbourhood structure loses 20–80% of its overlap. Fine-tuning reshuffles neighbours long before it changes the global spectral shape of the space. **H2 is not supported**, and global isotropy is a weak drift signal.

4.3 H3: Preservation tracks forgetting, but there is no 0.6 threshold

Overall association. Across all 348 non-zero checkpoints, lower preservation goes with more forgetting. Checkpoints with preservation ≥ 0.6 ($n = 260$) lose 2.7 pp GK accuracy on average, and 39% show a significant decline. Checkpoints below 0.6 ($n = 88$) lose 9.2 pp, and 82% show a significant decline (Mann–Whitney $p = 5 \times 10^{-12}$). Spearman ρ between preservation and ΔGK is 0.69, and between preservation and ΔWikiText-103 NLL it is -0.90.

Forgetting starts well above 0.6. Table 3 lists the first checkpoint with a significant GK decline for each run. In 16 of the 21 runs with a detected decline, forgetting begins while preservation is still ≥ 0.6, usually at 0.77–0.90. Qwen2.5-0.5B loses 8.8 pp GK after 3,000 medical steps and 9.9 pp after 3,000 WikiText-103 steps, while its preservation stays ≥ 0.71. At lr 5e-4 it loses 11.7 pp by step 500, at preservation 0.70. The only runs that start forgetting below 0.6 are the diverged Pythia-160M LoRA runs.

The mapping differs across families. At preservation ≈ 0.25, Qwen2.5-0.5B (lr 1e-3) has lost 30 pp GK, while Pythia-160M has lost only ∼4 pp (figure 4, left). Part of this gap comes from Pythia's MMLU accuracy being at chance (∼23%), so its GK signal comes only from ARC-Easy.

Table 3: First checkpoint with a significant drop in general-knowledge accuracy (McNemar $p < .05$ against step 0) and the preservation at that point. Only the Pythia-160M LoRA runs, all of which diverged, start forgetting below the hypothesised 0.6 threshold.

Run	Onset step	Preservation at onset
Qwen2.5-0.5B medical (seeds 1/2/3)	500 / 400 / 750	0.79 / 0.80 / 0.77
Qwen2.5-0.5B general	100	0.81
Qwen2.5-0.5B lr 5e-4 (medical / code)	50 / 300	0.82 / 0.81
Qwen2.5-0.5B lr 1e-3 (medical / code)	50 / 100	0.78 / 0.78
Qwen2.5-1.5B medical	50	0.90
Qwen2.5-3B medical	200	0.84
Pythia-410M (medical / code)	150 / 500	0.79 / 0.81
Pythia-160M LoRA (5 runs)	100–300	0.43–0.50
Pythia-160M full FT (code / medical)	1,500 / 2,500	0.73 / 0.67

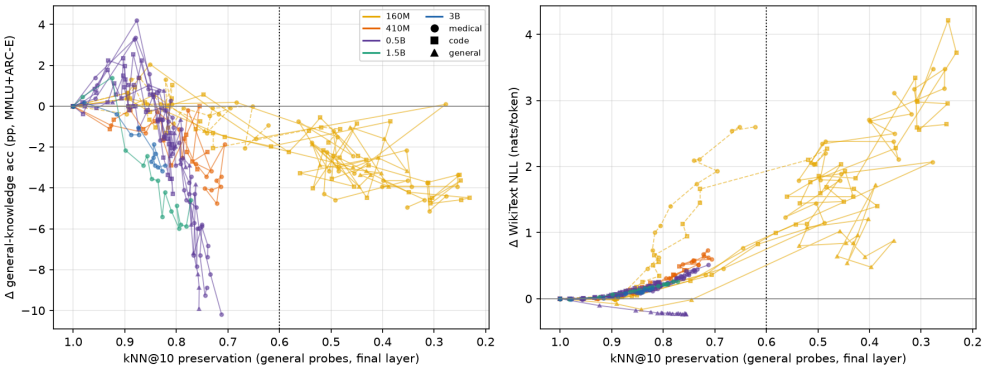


Figure 4: Preservation against forgetting for every checkpoint (colour: model; marker: domain; dashed: full fine-tuning). **Left:** change in GK accuracy. Qwen models (purple, green, blue) lose up to 10 pp while preservation is still above 0.7, whereas Pythia-160M (yellow) loses only a few points at 0.3. The dotted line marks the hypothesised 0.6 threshold. **Right:** change in WIKITEXT-103 NLL. The relation is tight and shared across models; the lower branch is the WIKITEXT-103 control run.

Still, the same preservation value means very different amounts of benchmark damage in the two families.

General-text perplexity is the exception. The relation between preservation and ΔWIKITEXT-103 NLL is tight and nearly the same for all models (figure 4, right). Losing neighbourhood structure on general text goes together with losing general-text language modelling. The one exception is the WIKITEXT-103 control, where NLL improves while preservation drops. **H3 is not supported**: preservation is associated with forgetting, but there is no onset at 0.6. For Qwen models, accuracy-based forgetting begins around 0.8.

4.4 H4: Code preserves more general geometry than medical text

The premise holds; the prediction fails. Clinical notes are much closer to WIKITEXT-103 than Python code is (JSD 0.45 vs. 0.63 bits; table 4). Yet over 144 matched checkpoints, medical fine-tuning preserves *less* general-text geometry than code (-0.043, medical higher at only 17% of checkpoints, $p = 3 \times 10^{-12}$). Medical fine-tuning also causes more forgetting: 1.9 pp more GK loss and 0.21 nats more WIKITEXT-103 NLL increase. All three Qwen2.5-0.5B seeds agree (final preservation 0.74 / 0.71 / 0.74 for medical vs. 0.82 / 0.81 / 0.81 for code), as do Qwen2.5-0.5B at lr 5e-4 (0.57 vs. 0.63) and 1e-3 (0.22 vs. 0.27), and Pythia-410M (0.71 vs. 0.74). Only Pythia-160M follows H4's direction (LoRA: 0.28 / 0.35 vs. 0.25 / 0.30; full fine-tuning: 0.62 vs. 0.51), and its LoRA runs diverged in both domains.

Table 4: Medical vs. code fine-tuning. **Top:** paired differences over 144 checkpoints matched on model, recipe, seed, and step (Wilcoxon signed-rank). Negative preservation differences mean medical preserves *less*. **Bottom:** vocabulary overlap of each corpus with WIKITEXT-103 ($\sim$2M Qwen tokens each). Medical text is much closer to general text, yet it preserves less general-text geometry.

Paired comparison	**Medical − code**	p
kNN@10 preservation	-0.043 (medical higher at 17%)	3×10^{-12}
Centred kNN@10 preservation	-0.044	3×10^{-14}
CKA	-0.011	2×10^{-11}
Δ GK accuracy	-1.9 pp (medical forgets more)	3×10^{-10}
Δ WIKITEXT-103 NLL	$+0.21$ nats (medical forgets more)	9×10^{-18}
Corpus overlap with WIKITEXT-103	**Medical**	**Code**
Unigram JSD (bits, $\downarrow$ closer)	**0.45**	0.63
Token mass in top-10k WIKITEXT-103 tokens	**67%**	36%
Word-type coverage	**90%**	80%

Two possible explanations. The first is *interference*. Clinical English uses the same tokens and features as general English, so adapting to it rewrites features that general text depends on, while code lives in a more separate region of the space. The second is *amount of change*. The base models already model this Python corpus well. Under the standard recipe, code fine-tuning gave no in-domain gain for Qwen2.5-0.5B (held-out NLL 1.58 $\rightarrow$ 1.68), while medical fine-tuning really adapted the model (2.34 $\rightarrow$ 1.98). Small, noise-like updates should move the geometry less. Our design cannot separate these two explanations, because the domain contrast is confounded with how much there is to learn. Either way, **H4 is not supported**, and vocabulary overlap does not predict geometric preservation here. As expected for real domain adaptation, under medical fine-tuning the medical probes move the most (Qwen2.5-0.5B: 0.64 for medical probes vs. 0.74 for general probes).

5 Discussion

Is collapse speed a property of model size? Only partly. Within a family and a fixed recipe, larger models keep more geometry and forget less, which agrees with prior reports that larger models forget less [Kalajdzievski, 2024]. But the number of steps until preservation falls below 50% is not a size constant. It depends mainly on update size (learning rate, LoRA vs. full fine-tuning) and on whether optimisation stays healthy. Under the standard recipe, healthy runs of models from 410M to 3B level off at 0.7–0.85 on a log-step axis. Extrapolating, a 7B model would very likely stay above 0.5 past 2,000 steps. That would make H1's 7B clause true but uninformative, since a 0.5B model already behaves this way. We did not measure 7B.

What is preservation good for? We see three uses and one non-use.

- It is a cheap, label-free proxy for damage to general-text language modelling, and its relation to ΔNLL is nearly the same across models ($\rho = -0.90$).
- A sharp early drop (e.g., below $\sim$0.6 within a few hundred steps) is a strong sign of over-aggressive or unstable fine-tuning. Every such run in our study also had rising in-domain loss.
- Because most drift happens in the first $\sim$100 steps, a monitor can flag a bad learning rate very early in training.
- It is not a calibrated alarm for benchmark forgetting. Qwen models lose MMLU and ARC-EASY accuracy at preservation 0.8–0.9, and part of the MMLU loss reflects losing the multiple-choice answer *format*: plain language-model training on WIKITEXT-103 costs Qwen2.5-0.5B 11 pp on MMLU. A practical monitor should combine the *rate* of preservation loss with a family-specific calibration, or track probe-set NLL directly.

Why do global and local geometry disagree? Fine-tuning reorders local neighbourhoods long before it changes the global spectrum of the space. Isotropy summarises the covariance of all probes, so it stays within a few percent while 20–80% of neighbour overlap is lost. Global isotropy statistics are therefore poor drift indicators.

Surprises. Three results ran against our expectations. First, in-distribution fine-tuning on WIKITEXT-103 drifts about as much as domain fine-tuning and causes the largest MMLU loss for Qwen2.5-0.5B. Second, Pythia-160M diverges under LoRA at lr 2e-4 in every domain, while full fine-tuning at lr 1e-5 learns stably. Third, fine-tuning Qwen2.5 on Python barely changes in-domain loss, because these base models are already strong on code.

5.1 Limitations

Scale coverage. We have no 7B results. A 7B run started on the only large GPU slice but ran out of memory when another user's job grew; at the measured $\sim$7 s per step plus 8.5 minutes per evaluation, it could not have finished. The Qwen2.5-1.5B and Qwen2.5-3B medical runs stop at 2,500 and 1,500 steps, and we have no 1.5B or 3B code runs, so H4 is tested only at $\leq$0.5B.

Recipe coverage. Each size uses one standard recipe, and the learning-rate and full fine-tuning ablations cover one model each. In practice larger models often get smaller learning rates, which would reduce their drift further.

Domain confound. Code fine-tuning did not lower in-domain loss, so the domain contrast is confounded with the amount of adaptation. A design that matches in-domain loss reduction across domains would separate the two explanations in section 4.4.

Seeds and noise. Only Qwen2.5-0.5B (three seeds) and Pythia-160M (two seeds) medical and code runs are replicated. Seed variation is small for Qwen (≤ 0.016) but reaches $\sim$0.09 for the diverging Pythia-160M runs. A single GK estimate on 1,070 items has about $\pm$2.5 pp sampling noise, which paired McNemar tests reduce but do not remove. MMLU is at chance for Pythia.

Probes and proxies. We use mean-pooled 128-token WIKITEXT-103 probes. Token-level or static-embedding neighbourhoods may behave differently, and under LoRA the input embedding matrix is frozen. WIKITEXT-103 stands in for pretraining data, and the medical corpus consists of LLM-written synthetic notes rather than real clinical text.

Broader impact. A cheap geometric monitor could reduce the compute spent on evaluation during fine-tuning. Our results warn against relying on one fixed threshold: a monitor tuned on one model family could miss substantial forgetting in another, which matters when fine-tuned models are deployed in sensitive domains such as medicine.

6 Conclusion

We tracked how fine-tuning reshapes the local neighbourhood structure of language-model representations across 22 runs, five model sizes, three domains, and 17 checkpoints. Fine-tuning changes neighbourhoods quickly, mostly within the first $\sim$100 steps, and larger models change less at matched steps. All four specific hypotheses fail. Collapse below 50% depends on the learning rate and on training stability more than on a sub-1B vs. 7B split. Isotropy never declines monotonically. Benchmark forgetting begins while preservation is still around 0.8, not at 0.6. Code fine-tuning, not medical, preserves more general-text geometry for most models, despite lower vocabulary overlap.

Takeaway: kNN preservation is a cheap proxy for damage to general language modelling and a reliable sign of unstable fine-tuning. It is not a universal forgetting threshold.

Future work should run 7B models and complete the 1.5B and 3B code runs on dedicated hardware, fit crossing time as a function of update norm with a learning-rate $\times$ size grid, and compare domains matched on in-domain loss reduction. A lead-time analysis would show whether probe-set NLL, CKA, or kNN preservation gives the earliest warning. Replay or L2-SP regularisation [Li et al., 2018] could test whether preservation can be decoupled from forgetting.

References

Dan Biderman, Jacob Portes, Jose Javier Gonzalez Ortiz, Mansheej Paul, Philip Greengard, Connor Jennings, Daniel King, Sam Havens, Vitaliy Chiley, Jonathan Frankle, Cody Blakeney, and John P. Cunningham. LoRA learns less and forgets less. *Transactions on Machine Learning Research*, 2024.

Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of ICML*, 2023.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? Try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

Kawin Ethayarajh. How contextual are contextualized word representations? Comparing the geometry of BERT, ELMo, and GPT-2 embeddings. In *Proceedings of EMNLP-IJCNLP*, 2019.

Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don't stop pretraining: Adapt language models to domains and tasks. In *Proceedings of ACL*, 2020.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *Proceedings of ICLR*, 2021.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *Proceedings of ICLR*, 2022.

Damjan Kalajdzievski. Scaling laws for forgetting when fine-tuning large language models. *arXiv preprint arXiv:2401.05605*, 2024.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *Proceedings of ICML*, 2019.

Sunjun Kweon, Junu Kim, Jiyoun Kim, Sujeong Im, Eunbyeol Cho, Seongsu Bae, Jungwoo Oh, Gyubok Lee, Jong Hak Moon, Seng Chan You, Seungjin Baek, Chang Hoon Han, Yoon Bin Jung, Yohan Jo, and Edward Choi. Publicly shareable clinical large language model built on synthetic clinical notes. In *Findings of ACL*, 2024.

Xuhong Li, Yves Grandvalet, and Franck Davoine. Explicit inductive bias for transfer learning with convolutional networks. In *Proceedings of ICML*, 2018.

Yun Luo, Zhen Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang. An empirical study of catastrophic forgetting in large language models during continual fine-tuning. *arXiv preprint arXiv:2308.08747*, 2023.

Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.

Amil Merchant, Elahe Rahimtoroghi, Ellie Pavlick, and Ian Tenney. What happens to BERT embeddings during fine-tuning? In *Proceedings of the Third BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP*, 2020.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

Jiaqi Mu and Pramod Viswanath. All-but-the-top: Simple and effective postprocessing for word representations. In *Proceedings of ICLR*, 2018.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Maithra Raghu, Justin Gilmer, Jason Yosinski, and Jascha Sohl-Dickstein. SVCCA: Singular vector canonical correlation analysis for deep learning dynamics and interpretability. In *Advances in NeurIPS*, 2017.

William Rudman, Nate Gillman, Taylor Rayne, and Carsten Eickhoff. IsoScore: Measuring the uniformity of embedding space utilization. In *Findings of ACL*, 2022.

Jarkko Venna and Samuel Kaski. Local multidimensional scaling. *Neural Networks*, 19(6–7):889–899, 2006.

A Additional Experimental Details

LoRA target modules. For Qwen2.5 we adapt `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`. For Pythia we adapt `query_key_value`, `dense`, `dense_h_to_4h`, and `dense_4h_to_h`.

Memory. To fit training into 2–5 GB GPU slices, the loss is computed over 256-token chunks with recomputation, together with gradient checkpointing.

Corpora. Each run uses a seeded prefix of ∼6.1M tokens from its corpus. Code files are 800–20k characters long and filtered to exclude auto-generated files.

Held-out in-domain NLL (nats per token, step 0 → step 3,000).

Table 5: Held-out in-domain NLL before and after fine-tuning under the standard recipe. Code fine-tuning gives no gain for Qwen2.5-0.5B, and Pythia-160M LoRA diverges in both domains.

Model	Code	Medical
Qwen2.5-0.5B (lr 2e-4)	1.58 → 1.68 (no gain)	2.34 → **1.98** (learned)
Pythia-410M	1.55 → 1.83	2.34 → 2.08
Pythia-160M LoRA	2.18 → 4.24 (diverged)	3.15 → 3.79 (diverged)

Word-type coverage and floors. As a floor for the vocabulary-overlap measures, a split-half comparison of WIKITEXT-103 with itself gives JSD 0.01 bits, 91% token mass in the top-10k tokens, and 97% word-type coverage.

Domain comparison figure. Figure 5 overlays all seed-1 runs by domain.

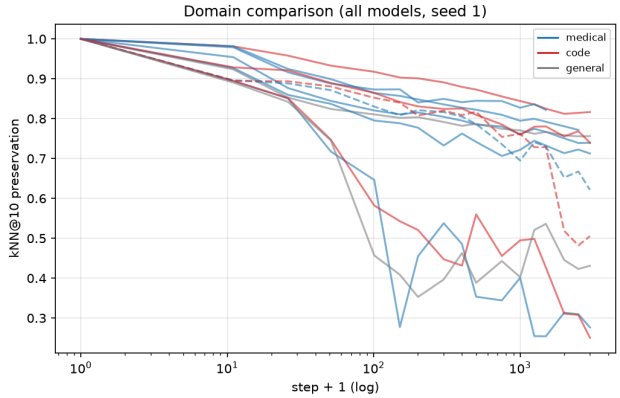


Figure 5: kNN@10 preservation for all models (seed 1), coloured by fine-tuning domain. Among the stable runs at the top, code (red) usually lies above medical (blue). The low cluster is the diverging Pythia-160M LoRA runs.

Software. Python 3.12.12, torch 2.14.1, transformers 5.20.2, peft 0.21.2, numpy 2.5.4, scipy 1.18.1, pandas 3.1.0. No external APIs were used.